

文件上传漏洞报告 (Lab1: 通过 Web Shell 上传远程代码执行)

1. 漏洞名称

危险文件上传 (Unrestricted File Upload) — Web Shell 远程代码执行 (RCE)

2. 漏洞等级

高危 (Critical / 致命)

3. 漏洞描述

该网站的“头像上传”功能存在严重的文件上传漏洞。后端在处理用户上传的文件时，**未对文件扩展名及文件内容进行任何有效验证**，导致攻击者可以直接上传包含恶意 PHP 代码的 Web Shell 脚本。

当攻击者通过浏览器直接访问上传后的 PHP 文件时，服务器会解析并执行其中的恶意代码，从而在服务器端读取敏感文件（如 `/home/carlos/secret`），实现远程代码执行。

4. 漏洞位置

文件上传点: POST `/my-account/avatar`

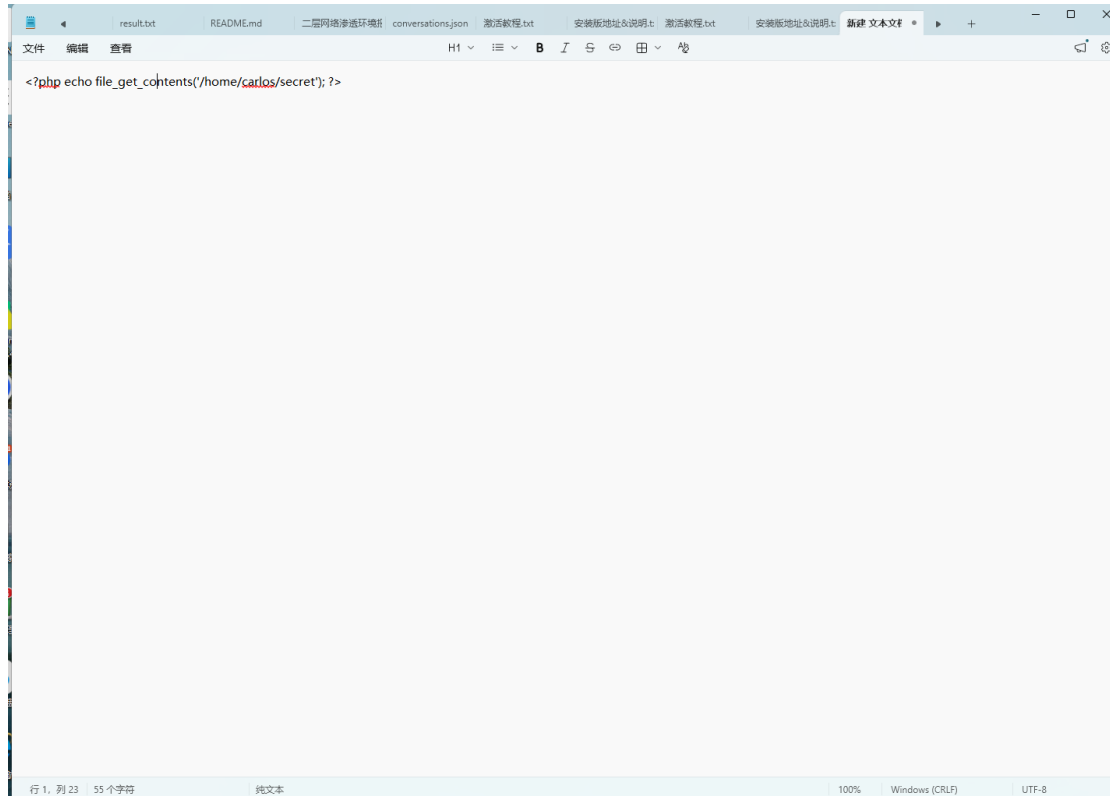
恶意文件访问点: GET `/files/avatars/exploit.php`

5. 漏洞复现过程

第一步: 编写恶意 Web Shell Payload

在本地创建一个名为 `exploit.php` 的文本文件，写入用于读取服务器敏感文件的 PHP 代码。

图 1: 编写读取服务器文件的 PHP Web Shell

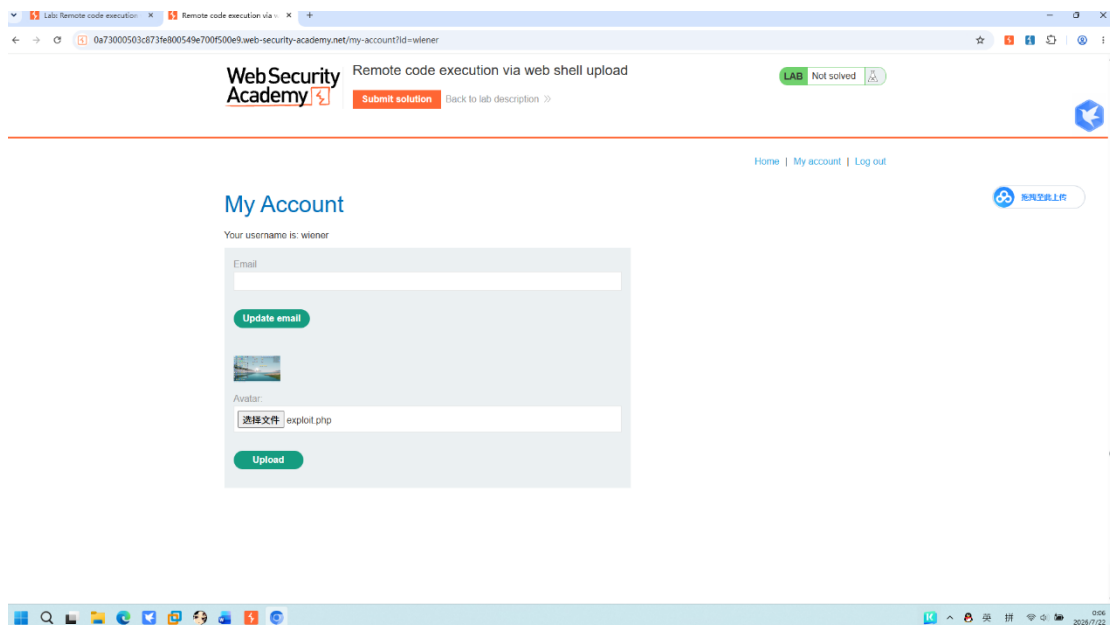


说明：利用 PHP 的 file_get_contents 函数，直接读取服务器上指定路径的敏感文件。

第二步：正常上传恶意 PHP 文件

登录账户，在“我的账户”页面的头像上传功能处，关闭 Burp 抓包，选择 exploit.php 文件并点击 Upload 按钮。

图 2：在浏览器端直接上传 Web Shell

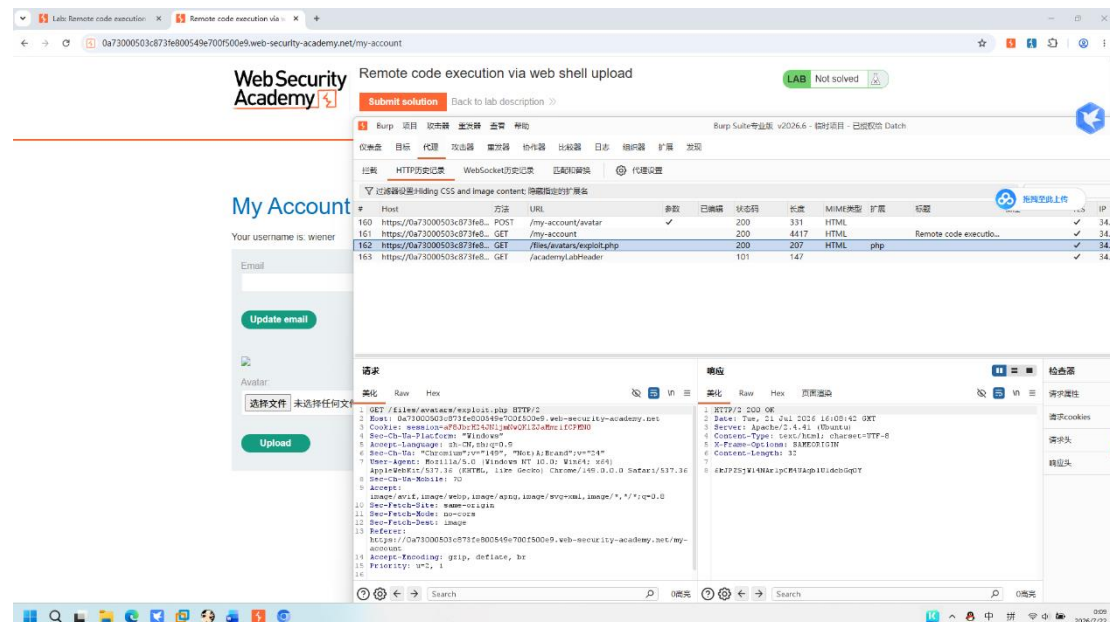


说明：由于后端无任何文件类型与内容校验，.php 文件被成功上传至服务器。

第三步：发现上传后的 Web Shell 访问路径

上传成功后，在 Burp Suite 的 HTTP History 中，观察图片资源的加载过程。发现所有的头像图片均通过 /files/avatars/[文件名] 进行加载。基于此规律，确定访问 Web Shell 的路径为 /files/avatars/exploit.php。

图 3：Burp HTTP History 中定位 Web Shell 访问请求

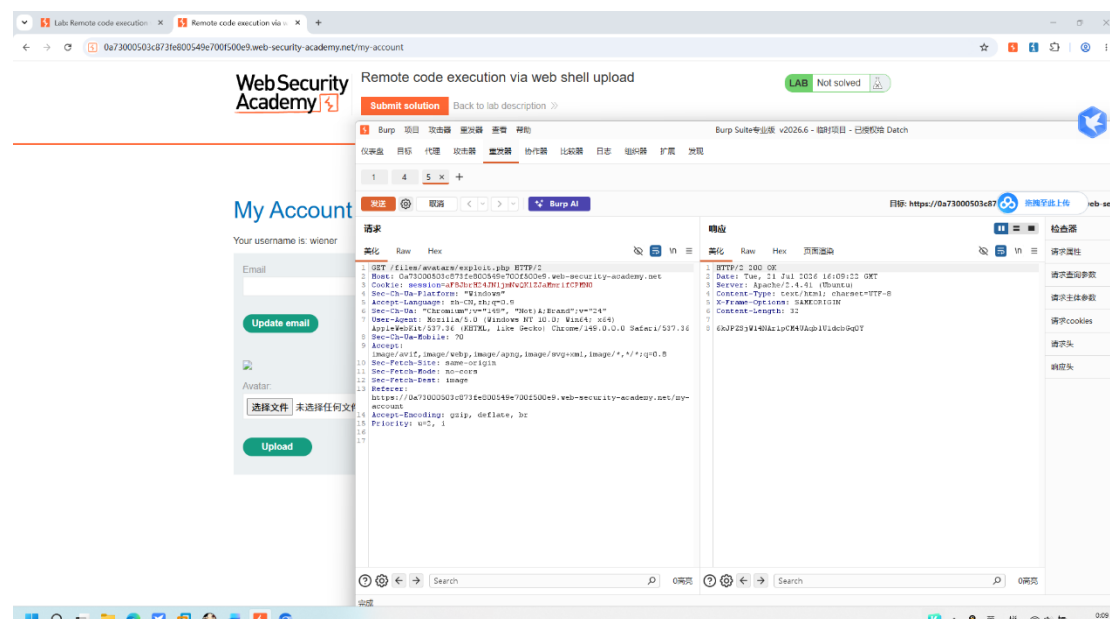


说明：Web 服务器没有将 PHP 文件作为纯文本返回，而是尝试解析执行了它。

第四步：触发 Web Shell 并获取敏感信息

将访问 Web Shell 的请求发送至 Burp Repeater，点击 Send 执行。

图 4：Burp Repeater 成功执行 Web Shell 并获取数据



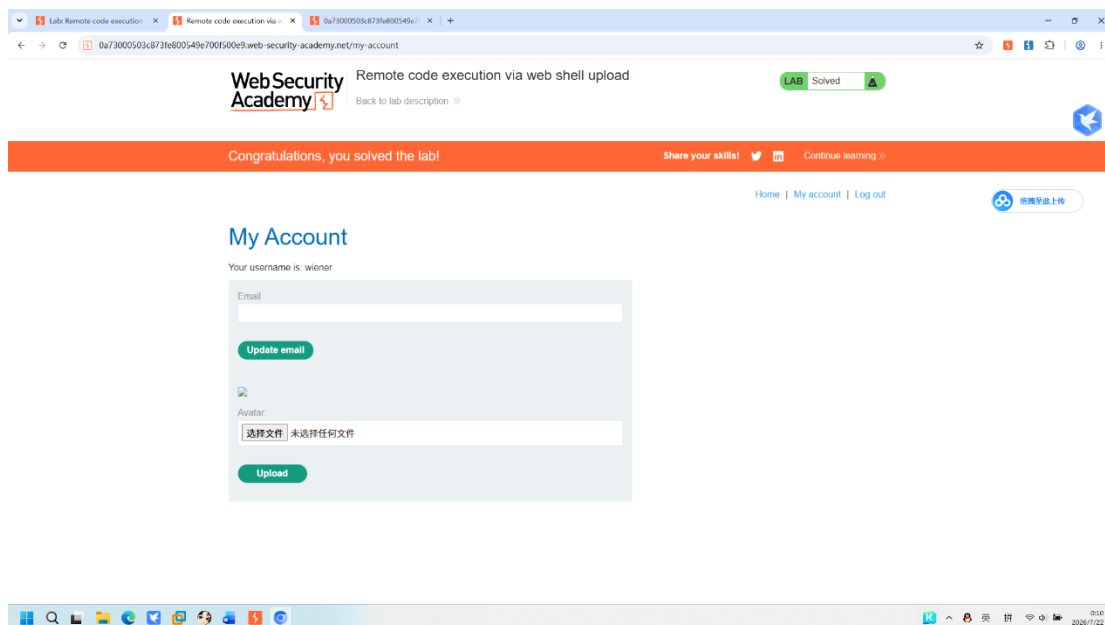
说明：Web 服务器解析了 exploit.php，执行

了 `file_get_contents('/home/carlos/secret')`，并将该文件的内容直接打印在 HTTP 响应体中。漏洞成功利用！

第五步：提交获取的 Secret，完成实验

复制 Repeater 中获取到的 Secret 内容，通过页面上的“Submit solution”按钮提交。

图 5：Web Shell 攻击成功，实验已解决



说明：靶场确认漏洞利用成功，实验完成。

6. 漏洞原理

该漏洞属于“不安全的文件上传”导致的“远程代码执行（RCE）”：

1. **防护缺失**：后端在处理文件时，没有校验文件的后缀名（黑名单/白名单失效），直接将 .php 文件上传到了 Web 可访问的目录。
2. **可预测的存储路径**：文件被保存在固定的 `/files/avatars/` 目录下，且文件名未做混淆处理。
3. **代码执行**：当 Web 服务器（如 Apache/Nginx）接收到对 .php 文件的请求时，会启动 PHP 解析器执行脚本内容，导致恶意系统函数被服务器执行。

7. 漏洞影响

- **远程代码执行（RCE）**：攻击者可上传功能更强的 Web Shell（如“中国菜刀”、“冰蝎”），通过浏览器轻松获取服务器的 Shell 控制权。
- **系统沦陷**：攻击者可以执行任意系统命令（如查看所有用户、下载数据库备份、甚至删除整个网站目录），导致服务器被完全接管。

8. 修复建议

- ① **严格的扩展名白名单校验**：必须限制上传的文件后缀，只允许 .jpg、.png、.gif，**严禁** .php、.phtml、.asp、.jsp 等脚本后缀。
- ② **内容与 MIME 校验**：服务器端必须校验上传文件的真实 MIME 类型（通过读取文件头 Magic Bytes，如图片必须为 FF D8），不能仅凭 HTTP Header 中的 Content-Type 判断。
- ③ **重命名与权限隔离**：上传后的文件应重命名为无规则的 UUID 字符串（去除后缀），且存放目录必须**禁止执行 PHP 解析权限**（如在 .htaccess 或 Nginx 中配置 `location ~ \.php$ { deny all; }`）。
- ④ **存储路径不可预测**：不要使用 avatars/ 这种规律明显的路径。

9. 工具

- **Burp Suite**（HTTP History 分析、Repeater 重放）
- **PortSwigger Web Security Academy**（靶场平台）